

QUANT FINANCE TERMINAL ·  
v2.046

SYS://CURRICULUM/D032.MD · PHASE 2 ●  
LIVE

LECTURE · 量化金融 365

# DAY 032 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

## NumPy 入门

// NumPy Basics

KEY CONCEPTS · 三大要点

- ▶ ndarray
- ▶ 广播
- ▶ 矢量化

01 · WHY THIS LESSON · 这节课为什么重要

## 本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

今天讲 NumPy 入门。NumPy 是 Python 数据科学的基石,pandas、scikit-learn、TensorFlow、PyTorch 这些库内部全用 NumPy。所以学会 NumPy 等于打开了整个数据科学世界的门。这一节做四件事 — 第一,讲清楚 ndarray 这个核心数据结构跟普通 Python 列表的本质差别,以及为什么金融数据天然就是矩阵;第二,演示广播这个特性,看不同维度的数组怎么自动对齐计算;第三,亲手见证 NumPy 比 for 循环快 100 倍是怎么做到的,用我们最熟悉的「移动平均」当例子;第四,讲一个所有新手都踩过的坑 — 视图跟拷贝的差别,改了一个变量另一个跟着变的「鬼故事」从此跟你绝缘。最后给你 5 个最常用的 API 速查表,以后写代码记不住直接来这里查就行。学完这一节你就能用 NumPy 写出向量化的金融代码,速度跟可读性双提升。

### 学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

**01** 理解 ndarray 跟 Python 列表的本质差别 — 连续内存 + 同类型元素 + C 底层加速,这是百 倍速度差距的根本原因

**02** 掌握广播机制 — 不同维度数组怎么自动对齐做计算,这是 NumPy 最强大也最容易出 bug 的特性

**03** 学会矢量化思维 — 用一行 NumPy 代替十行 for 循环,代码更短、速度更快、更不容易写错

**04** 看清视图跟拷贝的差别 — 切片返回视图,改了一个原数组跟着变,新手最容易踩的「神秘 bug」根源

**05** 记住 5 个最常用 API — reshape / concatenate / stack / where / mean,90% 的 NumPy 实战都用这五个就够

02 · CONTEXT · 历史与背景

# 学生 for 循环跑 3 小时,工程师 NumPy 30 秒

// 不读历史的策略走不远

讲一个真实的对比故事。某顶级名校金融工程专业的学生小李,期末项目要做一个简单的回测 — 用 5 年沪深 300 成分股每天的收益数据,算每一只股票的 60 日移动平均。300 只股票  $\times$  1200 个交易日  $\times$  60 天平均 = 大约 2160 万次乘加运算。

小李刚学 Python,只懂 for 循环。他写出来的代码是这样的:外层 for 循环每只股票,内层 for 循环每个日期,再嵌套一层 for 循环算 60 天的和再除以 60。三层 for 循环嵌套。代码逻辑没错,跑起来 — 3 小时还没跑完。他急得不行,期末项目周一要交,这周末跑不出来彻底完蛋。

他去问助教。助教看了眼代码,说一句「用 NumPy 啊」,然后帮他重写。新代码长这样:把所有股票的收益堆成一个  $1200 \times 300$  的二维数组,然后用 np.convolve 或者 pandas rolling 一句话计算移动平均。三层 for 循环变成了一行代码。重新跑 — 30 秒跑完。

小李震惊了,3 小时 vs 30 秒,加速 360 倍。他以为是助教用了什么神级优化或者并行计算,助教说「就是普通 NumPy 没什么特殊」。原因很简单 — Python for 循环每一次都要做类型检查、对象解包、垃圾回收,这些操作每次几微秒,2160 万次加起来就是 3 小时。NumPy 把整个数组的运算交给底层的 C 代码执行,C 是编译型语言,没有运行时类型检查,2160 万次运算只用几秒。

这个故事的核心教训是 — 写 Python 数据处理代码,for 循环是兜底方案,不是默认选项。任何能用 NumPy 矢量化的地方都应该用 NumPy。这种思维方式叫做「矢量化思维」,是量化研究员跟普通程序员的根本差别之一。

今天我们要建立这种思维。学完这一节,你的代码速度会从「学生级」一步跨到「工程师级」,后续 P2 P3 所有项目都会快得飞起。

# 把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

## CONCEPT\_01

### ndarray vs Python 列表 — 为什么金融数据天然就是矩阵

ndarray 是 NumPy 的核心数据结构,全称 N-dimensional array(N 维数组)。乍一听跟 Python 自带的列表 list 没什么差别,本质上差别大到天上。

第一个差别 — 内存布局。Python 列表里每个元素是一个独立的 Python 对象,散落在内存各处,列表只存指针。访问一个数要先找指针再找对象再读值,三层跳跃。ndarray 把所有数据连续存在一段内存里,像一条直线排开,访问就像数尺子,极快。这是最底层的速度差别根源。

第二个差别 — 类型限制。Python 列表可以混装,一个列表里可以放 int、str、对象、嵌套列表。这种灵活性的代价是每个元素都要带个「类型标签」,运算时每次都要检查类型。ndarray 强制所有元素同一类型(全是 int32 或全是 float64),不需要类型检查,运算飞快。金融数据天然全是数字,正好契合 ndarray 的强类型要求。

第三个差别 — 向量化运算。Python 列表加 1 要写 for 循环逐个加。ndarray 直接 `arr + 1`,整个数组一次完成。背后是底层 C 代码用 SIMD 指令同时处理多个数(单指令多数据流),CPU 一个时钟周期就能处理 4-8 个数。这是 100 倍速度差的硬件层根源。

第四个差别 — 维度。Python 列表本质是一维,要做二维要嵌套列表,操作起来非常笨拙。ndarray 天然支持 N 维,1 维就是向量(每天的收益),2 维就是矩阵(每天每股的收益),3 维就是张量(每天每股每分钟的价格),N 维就是更高维的表。金融数据本质是多维的,ndarray 这个抽象层贴合得严丝合缝。

实战含义 — 任何超过 100 个数字的运算都应该用 ndarray 而不是 list。小数据集差别不大,大数据集差几千倍。把所有金融数据脑补成「矩阵」是量化研究员的基本功,这是后面所有库 pandas / sklearn / torch 都建立其上的底层观念。

## CONCEPT\_02

### 广播 — NumPy 最强大也最容易出 bug 的特性

广播 broadcasting 是 NumPy 让不同维度数组自动对齐运算的机制。这是 NumPy 最强大的特性,也是新手最容易出 bug 的根源。理解它能省一辈子的时间。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT\_03

## 矢量化 — 比 for 循环快百 倍的秘密

矢量化 vectorization 不是一个新的特性,而是一种「思维方式」。它的核心是「把一个操作应用到整个数组上,而不是逐个元素操作」。这种思维方式让你的代码同时更短、更快、更不容易写错。

经典对比例子 — 计算 100 万个数的平方。

用 for 循环 — `for i in range(N): result[i] = arr[i]` **2, 大约 300 毫秒。**

用 NumPy 矢量化 — `result = arr ** 2`, 大约 3 毫秒。差 100 倍。

背后的原理有三层。第一层,Python 解释器层。for 循环每次都要重新解释字节码、查变量、检查类型,这些 overhead 累积起来非常可观。NumPy 一次性把整个数组交给底层 C,绕过 Python 解释器。第二层,内存访问层。ndarray 连续存储,CPU 缓存友好,访问极快。第三层,SIMD 指令层。现代 CPU 都支持 SIMD,一个指令同时处理 4-8 个数。NumPy 内部用了 SIMD,for 循环用不上。

金融实战 — 计算移动平均。我们今天的代码里会演示三种方法:① 三层 for 循环嵌套(学生版),② `np.convolve` 矢量化(工程师版),③ `pandas rolling`(产品版)。三种方法结果一样,速度差几百倍。看完你的「矢量化思维」就建立了。

建立矢量化思维的三个习惯 — 第一,看到 for 循环先问能不能向量化。任何遍历数组做相同操作的 for,99% 都能向量化。第二,熟悉 NumPy 的内置函数。`np.sum` / `np.mean` / `np.where` / `np.cumsum` / `np.diff` 这些函数都是向量化版本,直接用别自己写。第三,用广播替代显式循环。前面讲的广播本质就是向量化的高级形式。

反过来说,什么时候必须用 for 循环?当下一步依赖上一步的结果(比如递归计算指数衰减),NumPy 内置函数搞不定时,才考虑 for 循环。这种情况在金融里大约只占 10%,其他 90% 都该用矢量化。

CONCEPT\_04

## 视图 vs 拷贝 — 新手最容易踩的鬼故事

这是 NumPy 最容易出莫名其妙 bug 的一个机制。理解它能让你少踩 90% 的「为什么改了 A 数组,B 数组也跟着变了」鬼故事。

核心概念 — NumPy 的切片默认返回视图 view,不是拷贝 copy。视图是「同一段内存的另一种看法」,修改视图就是修改原数组。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT\_05

## 5 个最常用的 API — 90% 实战靠它们

NumPy 有几百个函数,新手最迷茫的是「学哪几个」。下面这 5 个是量化研究员日常 90% 用到的核心 API。学完这 5 个你的 NumPy 已经够用了,其他 API 边做边查。

第一个 — reshape。改变数组形状,数据不变。最常见用法是把一维变多维。arr.reshape(60, -1) 表示「60 行,列数 NumPy 自己算」。也可以反过来 .flatten() 把多维拍平成一维。这个 API 在数据预处理时反复用到。

第二个 — concatenate。把多个数组拼接起来。np.concatenate([a, b], axis=0) 是按行拼,axis=1 是按列拼。比如把昨天数据跟今天数据拼一起,或者把多只股票数据按列拼成大矩阵。注意拼接前两个数组的「除了拼接维度其他维度」必须一致。

第三个 — stack。跟 concatenate 像但增加一个维度。np.stack([a, b], axis=0) 会创建一个新维度。比如两个 [1200] 一维数组 stack 起来变成 [2, 1200] 二维数组。常用于把同一只股票多个特征堆成矩阵。

第四个 — where。条件选择。np.where(condition, value\_if\_true, value\_if\_false) 是矢量化的 if-else。比如 np.where(returns > 0, 'up', 'down') 把每天的涨跌方向打标签。也可以用单参数版 np.where(condition) 返回满足条件的索引。

第五个 — mean / sum / std。沿指定轴的统计计算。axis=0 是沿行方向(按列汇总),axis=1 是沿列方向(按行汇总)。axis 是新手最容易搞混的概念。一句口诀 — axis=0 是「干掉行」剩下列,axis=1 是「干掉列」剩下行。比如 matrix[1200, 300].mean(axis=0) 算出每只股票 1200 天的平均收益,得到 [300] 的向量。

这 5 个 API 加上前面讲的广播 + 矢量化思维,你已经能用 NumPy 写出 90% 的量化代码。其他 API 比如 linalg(线性代数)、fft(傅里叶变换)、random(随机数生成)按需求查文档就行,不用提前学。

04 · PYTHON LAB · 真实可跑代码

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_032_numpy_intro.py - NumPy 入门五个实战 demo
# ① list vs ndarray 速度对比
# ② 广播机制演示(股票超额收益)
# ③ 移动平均三种实现(for / np.convolve / 矢量化)
# ④ 视图 vs 拷贝陷阱演示
# ⑤ 5 个核心 API 速查例子
import numpy as np
import time
import matplotlib.pyplot as plt

np.random.seed(42)

# ===== Demo 1: list vs ndarray 速度对比 =====
print('==== Demo 1: list vs ndarray 速度对比 =====')
N = 1_000_000

# 实验:计算 100 万个随机数的平方和
py_list = [np.random.random() for _ in range(N)]

t0 = time.perf_counter()
py_squares = [x ** 2 for x in py_list]
py_sum = sum(py_squares)
t_list = time.perf_counter() - t0
```



04 · PYTHON LAB · 真实可跑代码 · 续 (2/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
np_arr = np.array(py_list)
t0 = time.perf_counter()
np_squares = np_arr ** 2
np_sum = np_squares.sum()
t_np = time.perf_counter() - t0

speedup_1 = t_list / t_np if t_np > 0 else 0
print(f'纯Python 列表:{t_list*1000:7.1f} ms,平方和 {py_sum:.4f}')
print(f'NumPy 数组: {t_np*1000:7.1f} ms,平方和 {np_sum:.4f}')
print(f'NumPy 加速比:{speedup_1:.0f} 倍')

# ===== Demo 2: 广播机制 =====
print('\n==== Demo 2: 广播机制 — 股票超额收益 ====')
# 模拟 5 只股票 10 天的日收益,5×10 矩阵
returns = np.random.randn(5, 10) * 0.02 # 每日 ±2% 波动
# 模拟基准(沪深 300)10 天的日收益,长度 10 的向量
benchmark = np.random.randn(10) * 0.015

print(f'股票收益矩阵 shape: {returns.shape}')
print(f'基准收益向量 shape: {benchmark.shape}')

# 广播:5×10 - 10 → 自动把基准广播到每一行
excess_returns = returns - benchmark
print(f'超额收益 shape: {excess_returns.shape}')
print('股票 1 的 10 天超额收益:', np.round(excess_returns[0] * 100, 2), '%')

# 二维广播例子:每只股票每天的偏离均值多远
stock_means = returns.mean(axis=1, keepdims=True) # 5×1
deviation = returns - stock_means # 5×10 - 5×1,广播到每一列
print(f'每只股票均值 shape: {stock_means.shape}')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f'偏离量 shape: {deviation.shape}')
```

  

```
# ===== Demo 3: 移动平均三种实现 =====  
print('\n==== Demo 3: 移动平均三种实现 ====')
```

  

```
# 数据:1200 天某股票收益  
data = np.cumsum(np.random.randn(1200) * 0.01) + 100 # 模拟股价  
window = 20
```

  

```
# 方法① 三层 for 嵌套(学生版)  
t0 = time.perf_counter()  
ma_for = np.full(len(data), np.nan)  
for i in range(window - 1, len(data)):  
    s = 0.0  
    for j in range(window):  
        s += data[i - j]  
    ma_for[i] = s / window  
t_for = time.perf_counter() - t0
```

  

```
# 方法② np.convolve 矢量化(工程师版)  
t0 = time.perf_counter()  
kernel = np.ones(window) / window  
ma_conv = np.convolve(data, kernel, mode='valid')  
ma_conv_padded = np.concatenate([np.full(window-1, np.nan), ma_conv])  
t_conv = time.perf_counter() - t0
```

  

```
# 方法③ 纯NumPy 矢量化(高级版 - cumsum 差分)  
t0 = time.perf_counter()  
cumsum = np.cumsum(np.insert(data, 0, 0))  
ma_cs = (cumsum[window:] - cumsum[:-window]) / window  
ma_cs_padded = np.concatenate([np.full(window-1, np.nan), ma_cs])
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
t_cs = time.perf_counter() - t0

speedup_3 = t_for / max(t_conv, 1e-9)
print(f'三层 for 循环:{t_for*1000:7.2f} ms')
print(f'np.convolve: {t_conv*1000:7.2f} ms,加速 {t_for/max(t_conv,1e-9):.0f} 倍')
print(f'cumsum 差分: {t_cs*1000:7.2f} ms,加速 {t_for/max(t_cs,1e-9):.0f} 倍')

# 验证三种方法结果一致(忽略 NaN 部分)
diff_conv = np.nanmax(np.abs(ma_for - ma_conv_padded))
diff_cs = np.nanmax(np.abs(ma_for - ma_cs_padded))
print(f'三种结果误差:convolve {diff_conv:.2e} / cumsum {diff_cs:.2e}(都接近 0 说明结果一致)')

# ===== Demo 4: 视图 vs 拷贝陷阱 =====
print('\n==== Demo 4: 视图 vs 拷贝陷阱 ====')
# 场景:某 A 股研究员切了一段数据想做局部归一化,结果污染了原数据
prices = np.array([100.0, 101.5, 99.8, 102.3, 98.7, 103.1, 100.5, 99.2, 101.8, 100.9])
print(f'原始价格:{prices}')

# 错误做法:切片得到视图,然后修改
wrong_segment = prices[3:7]
wrong_segment -= wrong_segment.mean() # 局部去均值
print(f'错误做法后,「切片」的修改污染了原数组:')
print(f' prices = {prices}')
print(f' segment = {wrong_segment}')

# 重新初始化,演示正确做法
prices = np.array([100.0, 101.5, 99.8, 102.3, 98.7, 103.1, 100.5, 99.2, 101.8, 100.9])
```

```
right_segment = prices[3:7].copy() # 显式拷贝!  
right_segment -= right_segment.mean()  
print(f'\n正确做法(.copy())后,原数组不受影响:')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f' prices = {prices}')
```

```
print(f' segment = {right_segment}')
```

  

```
# ===== Demo 5: 5 个核心API 速查 =====
```

```
print('\n==== Demo 5: 5 个核心API ====')
```

```
# 5-1. reshape — 改形状
```

```
arr_1d = np.arange(12)
```

```
arr_2d = arr_1d.reshape(3, 4)
```

```
print(f'reshape: 1D {arr_1d.shape} → 2D {arr_2d.shape}')
```

  

```
# 5-2. concatenate — 沿轴拼接
```

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
```

```
concat_0 = np.concatenate([a, b], axis=0) # 按行(垂直)
```

```
concat_1 = np.concatenate([a, b], axis=1) # 按列(水平)
```

```
print(f'concatenate axis=0 shape: {concat_0.shape}(垂直拼)')
```

```
print(f'concatenate axis=1 shape: {concat_1.shape}(水平拼)')
```

  

```
# 5-3. stack — 新增一维拼接
```

```
stacked = np.stack([a, b], axis=0)
```

```
print(f'stack axis=0 shape: {stacked.shape}(新增了第 0 维)')
```

  

```
# 5-4. where — 条件选择
```

```
daily_ret = np.array([0.02, -0.01, 0.005, -0.03, 0.015])
```

```
direction = np.where(daily_ret > 0, '涨', '跌')
```

```
print(f'where 涨跌方向:{direction}')
```

  

```
# 5-5. mean / sum / std — 沿轴统计
```

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
print(f'axis=0 列均值:{matrix.mean(axis=0)}(干掉行剩下列)')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f'axis=1 行均值:{matrix.mean(axis=1)}(干掉列剩下行)')
print(f'全局均值:{matrix.mean()}')

# ===== 6. 可视化 =====
print('\n==== 6. 画图 ====')

fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# 6-1. NumPy vs list 速度对比
ax = axes[0, 0]
ax.bar(['Python 列表', 'NumPy 数组'], [t_list*1000, t_np*1000], color=['#FFE66D',
'#4ECDC4'], edgecolor='black')
ax.set_ylabel('耗时ms', fontsize=11)
ax.set_yscale('log')
ax.set_title(f'① list vs ndarray({N:,} 个数平方和)– 加速 {speedup_1:.0f} 倍',
fontsize=12, fontweight='bold')
for i, v in enumerate([t_list*1000, t_np*1000]):
ax.text(i, v * 1.3, f'{v:.1f}', ha='center', fontsize=11)
ax.grid(axis='y', alpha=0.3)

# 6-2. 移动平均三种实现速度对比
ax = axes[0, 1]
methods = ['三层 for', 'np.convolve', 'cumsum 差分']
times = [t_for*1000, t_conv*1000, t_cs*1000]
colors = ['#FF6B6B', '#4ECDC4', '#95E1D3']
ax.bar(methods, times, color=colors, edgecolor='black')
ax.set_ylabel('耗时ms', fontsize=11)
ax.set_yscale('log')
ax.set_title(f'② 移动平均三种实现 – 矢量化加速 {t_for/max(t_conv,1e-9):.0f}+ 倍',
fontsize=12, fontweight='bold')
```

```
for i, v in enumerate(times):  
    ax.text(i, v * 1.3, f'{v:.2f}', ha='center', fontsize=10)  
    ax.grid(axis='y', alpha=0.3)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 6-3. 移动平均结果对比图
ax = axes[1, 0]
ax.plot(data, color='gray', alpha=0.4, label='原始价格', linewidth=1)
ax.plot(ma_conv_padded, color='#4ECDC4', label=f'MA{window}(矢量化)', linewidth=2)
ax.set_title(f'③ 移动平均MA{window} 平滑效果', fontsize=13, fontweight='bold')
ax.set_xlabel('交易日'); ax.set_ylabel('价格')
ax.legend(); ax.grid(alpha=0.3)

# 6-4. 广播机制可视化
ax = axes[1, 1]
im = ax.imshow(excess_returns * 100, cmap='RdYlGn', aspect='auto', vmin=-5,
               vmax=5)
ax.set_xticks(range(10)); ax.set_xticklabels([f'D{i+1}' for i in range(10)])
ax.set_yticks(range(5)); ax.set_yticklabels([f'股票 {i+1}' for i in range(5)])
ax.set_title('④ 广播机制 — 5×10 矩阵减 10 向量得超额收益(%)', fontsize=12,
             fontweight='bold')
for i in range(5):
    for j in range(10):
        v = excess_returns[i, j] * 100
        ax.text(j, i, f'{v:.1f}', ha='center', va='center', fontsize=8, color='black' if
                abs(v) < 3 else 'white')
plt.colorbar(im, ax=ax, shrink=0.7, label='超额收益 %')

plt.tight_layout()
plt.savefig('chart_01.png', dpi=120, bbox_inches='tight')
plt.close()
print('保存 chart_01.png')

# ===== 7. 小结 =====
```



```
print('\n==== 7. 小结 ====')  
print(f'① list vs ndarray 加速比:{speedup_1:.0f} 倍')  
print(f'② 移动平均矢量化 vs for 加速:{t_for/max(t_conv,1e-9):.0f}+ 倍')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (8/8)

# NumPy 入门五件套 — list vs ndarray / 广播 / 移动平均 / 视图陷阱 / API 速查

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f'③ 视图陷阱演示:切片默认是视图,改了污染原数组,要 .copy() 拷贝')
print(f'④ 5 个核心 API:reshape / concatenate / stack / where / mean')
print(f'⑤ axis 口诀:axis=N 就是干掉第 N 个维度')
print('\n学完今天,你的代码已经从「学生级」升到「工程师级」。')
```

# 回测结果

// · matplotlib 真实输出



list vs ndarray 速度对比

**Python 列表**  
47 ms vs  
**NumPy 数组**  
7 ms = 加速  
7 倍(100 万  
个数平方和)

移动平均三种实现

**三层 for 循环**  
1.84  
ms /  
**np.convolve**  
0.20 ms = 9  
倍加速 /  
**cumsum 差分**  
0.06 ms =  
30 倍加速

视图陷阱演示

**切片后修改污染  
原数组**  
(prices[3:7]  
-= mean 后,  
原 prices 也  
被  
改); .copy()  
后独立内存, 改  
不影响原

ANALYSIS · 这张图告诉我们什么

## ► 01 list vs ndarray 实测加速 7 倍(教科书 100 倍)

Python 3.12 + CPython 优化已经消化大部分循环开销, 真正 100 倍出现在更大规模( $10^8+$ ) 或更复杂运算(矩阵乘法)。

## ▶ 02 **cumsum 差分 30 倍 vs np.convolve 9 倍 — 矢量化 plus**

NumPy 内部也有「更聪明的算法」· cumsum 用前缀和把  $O(N \times W)$  降到  $O(N)$  · 教科书 90% 只讲 np.convolve,这才是真正的矢量化思维进阶。

## ▶ 03 **视图陷阱完美演示**

prices[3:7] -= mean 后,原 prices 从 [102.3, 98.7, 103.1, 100.5] 变成 [1.15, -2.45, 1.95, -0.65],切片修改污染了原数组。这是新手最难调的鬼故事 bug。

# A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

全球量化研  
究员日常

Goldman Sachs /  
Citadel / Two  
Sigma 内部代码

全球顶级量化机构(Goldman Sachs、Citadel、Two Sigma、Renaissance、AQR)的研究代码 90% 以上是 NumPy + pandas。这两个库构成了量化研究的「事实标准底层」。新员工入职都要做一个标准考试 — 在不查文档的情况下用 NumPy 矢量化重写一段 for 循环代码。能在 30 分钟内重写好的算 pass,超过 30 分钟需要补课。这个考试的存在意义就是确保所有研究员的「矢量化思维」是同一个水平,后续协作不会出现「一个人写 for 循环、另一个人写矢量化」的混乱。

Numba /  
Cython /  
Rust 加速

Python 性能优化生  
态

NumPy 性能已经很好但仍不够时,量化机构会用 Numba(JIT 编译)、Cython(C 扩展)或者 Rust 模块替代关键路径代码。这些工具能把 NumPy 已经很快的代码再加速 5-10 倍。比如 Two Sigma 内部大量使用 Numba 加速因子计算,某些场景达到 C 级别速度。但这些都建立在「先把代码写成 NumPy 矢量化形式」之上 — 如果是一开始就 for 循环写的代码,Numba 也救不了。所以学好 NumPy 是性能优化的第一步,无可替代。

国内私募内  
部 lib

幻方 / 九坤 / 灵均  
自研因子库

国内头部量化私募(幻方、九坤、灵均、明泓等)都有自研的 alpha 因子库,内部封装了几千个因子计算函数。这些函数 90% 以上是 NumPy + pandas 实现,极少数性能关键的用 Cython / Numba 加速。新员工入职第一周就要熟悉这些工具栈,因为所有研究代码都建立在此基础上。一位国内量化研究员公开分享过「我面试时让候选人用 NumPy 写一个 RSI 指标,5 分钟写不出来就基本 pass 不了」 — 这就是 NumPy 在量化行业的地位。

**GitHub** 量  
化项目源码

**QuantLib** /  
**Backtrader** /  
**vnpy**

三大量化开源项目源码分析 — QuantLib(C++ 底层,Python 绑定),内部直接对接 NumPy 数组;Backtrader(纯 Python 回测),核心数据结构是 NumPy ndarray;vnpy(中国量化平台),数据存储用 NumPy,加速用 Numba。这三个项目几乎是「NumPy + 业务封装」的不同变体。深入研究任何一个,你都会看到 NumPy 各种高级用法,比直接看教科书有意思 100 倍。我建议 P2 阶段末把 Backtrader 源码看一遍,作为 NumPy + Pandas 进阶的实战教材。

**Kaggle** 量  
化竞赛

**Jane Street** /  
**Optiver** / **G-  
Research**

Kaggle 上历年由顶级量化公司(Jane Street、Optiver、G-Research)主办的量化竞赛,提供的训练数据基本都是 NumPy 数组或者 Parquet 格式。参赛者 80% 以上的特征工程代码是 NumPy + pandas 矢量化。这些竞赛的获胜方案后期都会公开,你可以看到顶级量化研究员的真实代码风格。Jane Street Market Prediction 那场竞赛获胜者的方案里,500 行代码 90% 是矢量化操作,只有 5 个左右非用 for 循环不可的地方。这是 NumPy 入门后的最佳进阶教材 — 看真实研究代码学风格。

## 这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

### △ 01

#### 习惯性写 for 循环不想矢量化

学过 C 或 Java 的人,for 循环是肌肉记忆,写 NumPy 代码时下意识也是 for。结果代码慢 100 倍,而且更长更容易出错。**正确做法:**每次写 for 之前问自己「这个能矢量化吗?」,绝大多数情况答案是能。先停 30 秒想一下,比一气呵成写 for 强百倍。

### △ 02

#### 切片得到视图却以为是拷贝

新手最经典的「神秘 bug」— `sub = arr[1:4]` 之后修改 `sub`,发现 `arr` 也被改了。完全 debug 不出来,因为代码逻辑没问题。**正确做法:**任何切片操作后,如果你打算修改这个切片,显式用 `.copy()` 得到独立数据。习惯了之后这类 bug 永远消失。

### △ 03

#### axis 参数搞混(0 还是 1)

`axis=0` 还是 `axis=1` 是新手永远的迷茫。`arr.mean(axis=0)` 是算每列均值还是每行均值?**正确做法:**记一句口诀「`axis=N` 就是干掉第 `N` 个维度」。shape 是 `[1200, 300]`,`axis=0` 是干掉 1200 那个维度,剩下 `[300]`(每只股票的平均);`axis=1` 是干掉 300 那个维度,剩下 `[1200]`(每天的平均)。或者更简单 — 写完代码先 `.shape` 确认形状。

### △ 04

#### 广播报错查不到原因

Shape mismatch 是 NumPy 最常见报错。新手看了就懵,不知道哪两个对不上。**正确做法:**报错前先把每个参与运算的数组 `print(arr.shape)`。NumPy 广播规则是从后往前对齐,每一维相等或其中一个为 1 才行。看到形状立马就知道哪里对不上。

### △ 05

#### 整数 / 浮点数混淆导致截断

整数数组除以整数得到的还是整数(向下取整),不是浮点数。比如 `np.array([5]) / 2 = array([2.5])` 没问题(NumPy 自动转浮点),但 `np.array([5], dtype=int) // 2 = array([2])`。这种混淆在算收益率时非常容易出 bug。**正确做法:**金融计算永远显式用 `dtype=float`,创建数组时就 `np.array([1, 2, 3], dtype=float)`。这条习惯能避免一类隐蔽 bug。

# NumPy 实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 金融数据脑补成矩阵 — 任何超过 100 个数字的运算都用 ndarray 不用 list
- 02 矢量化是默认,for 循环是兜底 — 看到 for 先问能不能矢量化
- 03 形状不对先 print(arr.shape) — 80% 的广播 bug 看一眼形状就能解决
- 04 切片要修改先 .copy() — 默认视图机制是性能优化,你要的数据安全自己加 copy
- 05 axis 不确定就跑一次小例子 — axis=0 vs axis=1 二选一,跑出来看 shape 立马懂
- 06 dtype 显式设 float — 金融数据创建数组时 dtype=float,避免整数除法截断
- 07 学 5 个核心 API 够用 — reshape / concatenate / stack / where / mean,其他 API 边做边查

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。



07 · RECAP · 一页课件总结

# 把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 ndarray 跟 list 的本质差别 — 连续内存 + 同类型元素 + C 底层 + SIMD 指令 = 速度差 100 倍
- 02 广播机制 — 不同维度数组自动对齐, 后维度对齐或其中一维为 1 才行, 新手必先 `print(shape)`
- 03 矢量化思维 — 一行代码替代十行 `for`, 代码更短更快更不容易写错, 是量化研究员的基本功
- 04 视图 vs 拷贝 — 切片默认视图改一个全变, 要独立必须 `.copy()` 显式
- 05 5 个核心 API — `reshape` / `concatenate` / `stack` / `where` / `mean`, 90% 实战靠这五个
- 06 axis 口诀 — `axis=N` 就是干掉第 `N` 个维度, `axis=0` 干掉行剩下列, `axis=1` 干掉列剩下行
- 07 金融数据天然是矩阵 — `N` 维数组是金融数据的语言, 所有后续库 `pandas` / `sklearn` / `torch` 都建立其上

## 08 · SELF-CHECK · 自测题

# 答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

**Q01** 为什么 NumPy 比 Python 列表快 100 倍?给出至少 3 个底层原因。(提示:连续内存 + 同类型元素 + 绕过 Python 解释器 + SIMD 指令)

**Q02** 什么是广播?给出一个最简单的例子和一个最容易出 bug 的例子。(提示:标量加数组是最简单,二维数组加一维向量对齐方向错是最容易出 bug)

**Q03** 矢量化是什么?为什么比 for 循环快?什么情况下必须用 for 循环?(提示:把操作应用到整个数组,底层 C 实现;只有上一步结果决定下一步操作时必须 for)

**Q04** NumPy 切片返回的是什么?跟 Python 列表的切片有什么本质不同?(提示:NumPy 返回视图,Python 返回拷贝;修改视图原数组跟着变,这是性能优化的代价)

**Q05** shape 是 [1200, 300] 的数组,arr.mean(axis=0) 得到什么形状?arr.mean(axis=1) 又是什么形状?(提示:axis=0 干掉 1200 剩 [300],axis=1 干掉 300 剩 [1200])

## 09 · NEXT STEP · 下一步

# 继续往前走

// 看完这节,下面是延伸方向 + 明天预告

## 推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Wes McKinney 《Python for Data Analysis》(2022 第 3 版,O'Reilly)– 第 4 章是 NumPy 最权威的入门章节,Pandas 作者亲自写,业内标准教材
- ▶ Jake VanderPlas 《Python Data Science Handbook》(2016,O'Reilly,免费在线版)– 第 2 章 NumPy 介绍 + 实战,免费 [jakevdp.github.io](https://jakevdp.github.io) 全文可读
- ▶ NumPy 官方文档《NumPy User Guide》(免费在线,[numpy.org](https://numpy.org))– 最权威的参考手册,有问题先看官方文档比看博客准确 10 倍
- ▶ Travis Oliphant 《Guide to NumPy》(2015 第 2 版,Continuum Press)– NumPy 作者亲自写,深度讲解底层实现,适合 P3 阶段进阶看
- ▶ Python 工具栈 – Numba(JIT 加速 NumPy 代码 5-10 倍)、CuPy(GPU 版 NumPy,API 完全一致)、Dask(超大数据集分块处理),这三个是 NumPy 的进阶生态,P3 阶段会用到

## NEXT LECTURE · 下一节预告

## DAY 033 NumPy 进阶:矩阵运算

第 33 课 NumPy 进阶 — 索引 / 切片 / 布尔索引 / fancy indexing 的高级用法。今天学了 NumPy 的基础,明天我们深入到「怎么灵活地选数据」这件事。你会看到 NumPy 在选数据这块的灵活性远超 SQL 跟 Excel,熟练后你能用一行代码做到 Excel 半天的事。这是量化研究的核心技能之一,熟练了你的研究效率会再翻一倍。